Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

Chapter 8: Approximation Algorithms

Algorithms Analysis and Design (CO3031)

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: chauvtn@hcmut.edu.vn

Semester 2 – 2022-2023

Course Learning Outcomes

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.

2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies*.

3. Able to discuss NP-completeness.

References

[1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.

[2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, *3rd Edition*, Pearson, 2012.

[3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.

[4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, TheBenjamin/Cummings Publishing, 1993.

Course Outline

- Chapter 1. Fundamentals
- Chapter 2. Divide-and-Conquer Strategy
- Chapter 3. Decrease-and-Conquer Strategy
- Chapter 4. Transform-and-Conquer Strategy
- Chapter 5. Dynamic Programming and Greedy Strategies
- Chapter 6. Backtracking and Branch-and-Bound
- Chapter 7. NP-completeness
- **Chapter 8. Approximation Algorithms**

Chapter 8. Approximation Algorithms

- 8.1. Why approximation algorithms?
- 8.2. Vertex Cover problem
- 8.3. Set Cover problem
- 8.4. Traveling Salesman problem
- 8.5. Scheduling Independent tasks
- 8.6. Bin Packing problem

8.1. Why approximation algorithms?

- Many problems of practical significance are NP-complete but are too important to abandon merely because obtaining an optimal solution is *intractable*.
- If a problem is NP-complete, we are unlikely to find a polynomial time algorithm for solving it exactly, but it may still be possible to find *near-optimal solution* in polynomial time.
- In practice, near-optimality is often good enough.
- An algorithm that returns near-optimal solutions is called an *approximation algorithm*.

8.1. Why approximation algorithms?

- Performance bounds for an approximation algorithm
  - i is an instance of an optimization problem.
  - $c(i)$ is the cost of a solution produced by the approximation algorithm and $c^*(i)$ is the cost of an optimal solution.
  - For a *minimization* problem, a performance bound is defined as: $c(i)/c^*(i)$.
  - For a *maximization* problem, a performance bound is defined as: $c^*(i)/c(i)$.
  - A performance bound for an approximation algorithm is expected to be *as small as possible*.

8.1. Why approximation algorithms?

- Performance bounds for an approximation algorithm
  - An approximation algorithm for the given optimization problem instance *i*, has a ratio bound of $p(n)$ if for any input of size n, the cost c of the solution produced by the approximation algorithm is within a factor of $p(n)$ of the cost c* of an optimal solution. That is:

$$\max(c(i)/c^*(i), c^*(i)/c(i)) \leq p(n)$$

  - Note that $p(n)$ is always greater than or equal to 1.
  - If $p(n) = 1$ then the approximation algorithm is an optimal algorithm.
  - The larger $p(n)$, the worse algorithm.

8.1. Why approximation algorithms?

- **Relative error**
  - We define the *relative error* of an approximation algorithm for any input size as

$$|c(i) - c^*(i)| / c^*(i)$$

  - We say that an approximation algorithm has a **relative error bound** of $\varepsilon(n)$ if

$$|c(i)-c^*(i)|/c^*(i) \leq \varepsilon(n)$$

  - Note that $\varepsilon(n)$ is always greater than or equal to 0.
  - If $\varepsilon(n) = 0$ then the approximation algorithm is an optimal algorithm.
  - The larger $\varepsilon(n)$, the worse algorithm.

8.2. Vertex Cover problem

- Vertex cover: given an undirected graph G=(V,E), a vertex cover is a subset $V' \subseteq V$ such that if $(u,v) \in E$, then $u \in V'$ or $v \in V'$ (or both).
- Size of a vertex cover: the number of vertices in it.
- The vertex cover problem: find a vertex cover of a minimum size.
- This problem is NP-hard since the related decision problem is NP-complete.

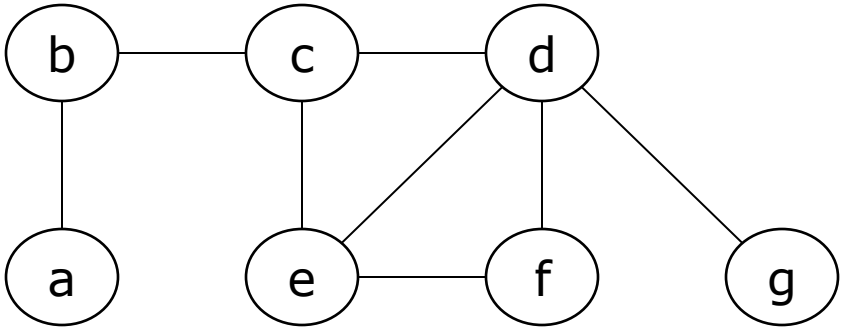
8.2. Vertex Cover problem

- Approximation algorithm for the vertex cover problem with time complexity of O(|E|)

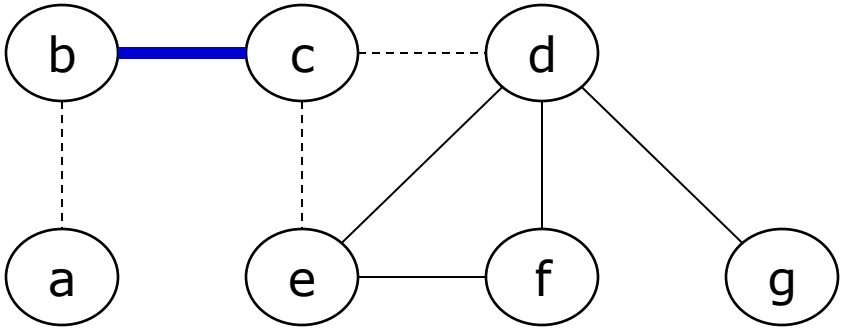
```
APPROX-VERTEX-COVER(G)
1  C ← ∅
2  E' ← E[G]
3  while E' ≠ ∅
4        do let (u, v) be an arbitrary edge of E'
5           C ← C ∪ {u, v}
6           remove from E' every edge incident on either u or v
7  return C
```

8.2. Vertex Cover problem

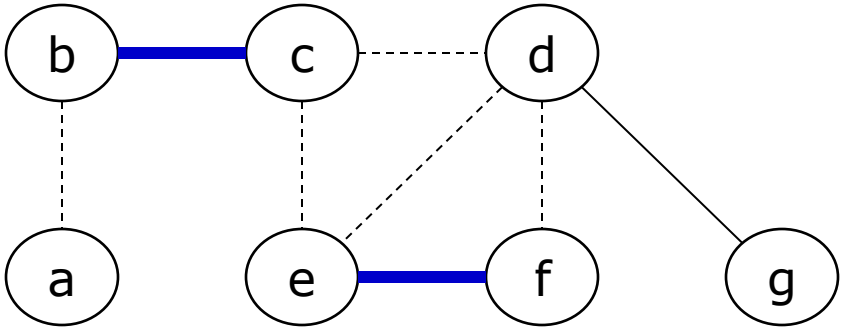
- Given graph (G) as follows, find its vertex cover of size as small as possible.



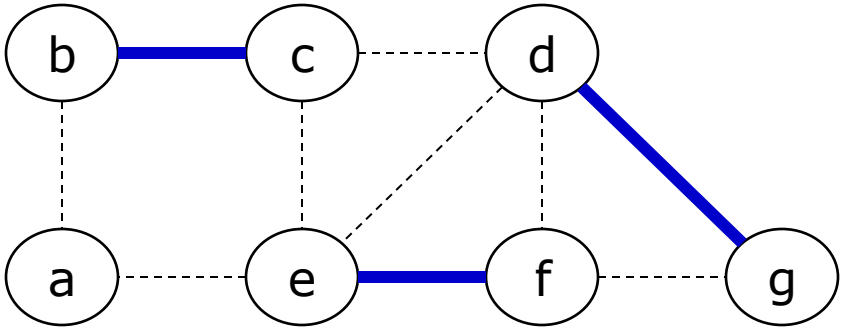
1. (G)



2. (G) after (b,c) is removed



3. (G) after (e,f) is removed



4. (G) after (d,g) is removed

A resulting vertex cover C = {b, c, e, f, d, g} vs. optimal C^* = {b, d, e}

8.2. Vertex Cover problem

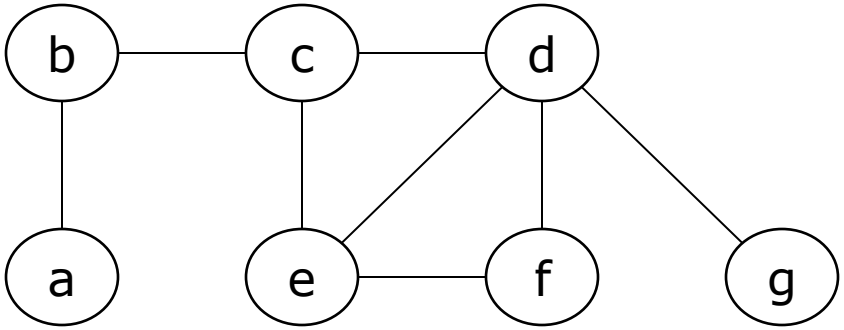
- Theorem:

  APPROX-VERTEX-COVER has a *ratio bound of 2*, i.e., the size of returned vertex cover set is at most twice the size of optimal vertex-cover.

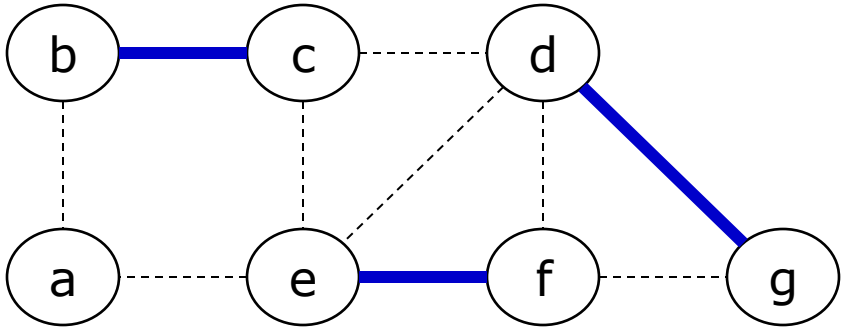
- Proof:
  - It runs in polynomial time O(|E|).
  - The returned C is a vertex-cover.
  - Let A be the set of edges picked in line 4 and C* be the optimal vertex-cover.
    - Then C* must include at least one end of each edge in A and no two edges in A are covered by the same vertex in C*, so $|A| \leq |C^*|$.
    - Moreover, $|C| = 2|A|$, so $|C| \leq 2|C^*|$.

8.2. Vertex Cover problem

- Given graph (G) as follows, find its vertex cover of size as small as possible.



1. (G)



4. (G) after (d,g) is removed

A resulting vertex cover C = {b, c, e, f, d, g} vs. optimal C^* = {b, d, e}

A = {(b,c), (e,f), (d,g)}

C = {b, c, e, f, d, g}

C* = { b, d, e}

$\rightarrow |C| = 2|A|$

$|A| \leq |C^*|$

$\rightarrow |C| \leq 2|C^*|$

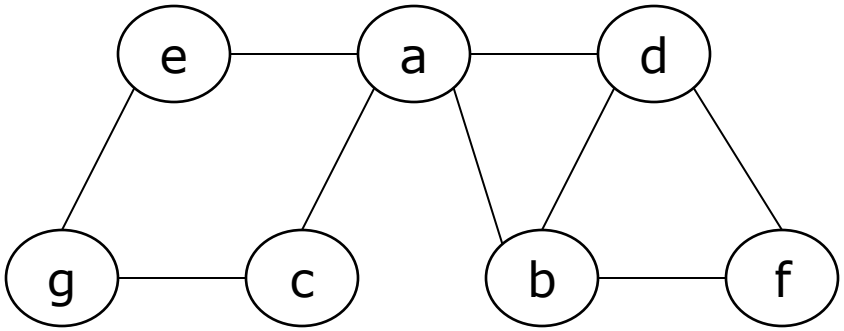
$\rightarrow |C|/|C^*| \leq 2$

8.2. Vertex Cover problem

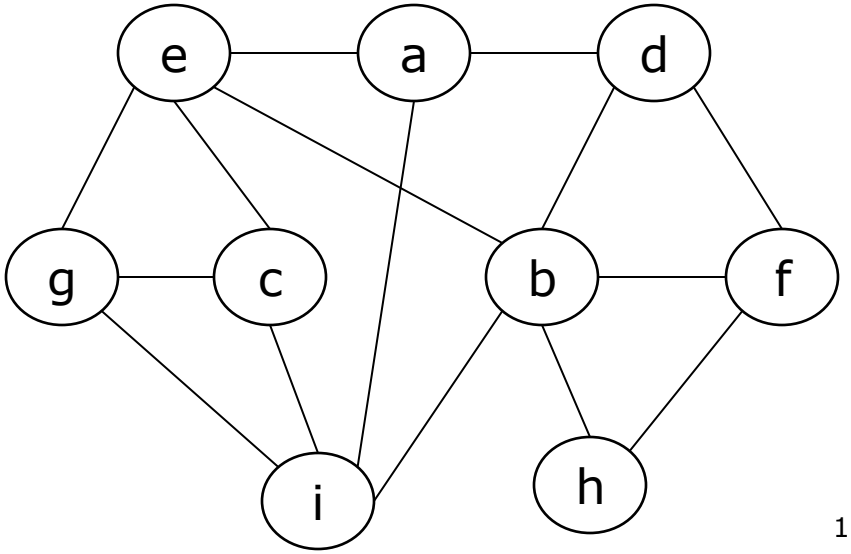
Your turn:

- Given the following graphs, find their vertex covers with the approximation algorithm. How different are they from the optimal covers?

8.2.1.



8.2.2.



8.2. Vertex Cover problem

Your turn:

- 8.2.3. Design and analyze another approximation algorithm for the vertex cover problem. Hint: you might think of heuristics by considering vertex instead of edge.
- 8.2.4. Trace-by-hand your algorithm in 8.2.3 for the graphs given in 8.2.1 and 8.2.2. Are your results better than the previous ones?

8.3. Set Cover problem

- The set cover problem is an optimization problem that models many resource-selection problems.
- An instance (X, F) of the set-cover problem consists of a finite set X and a family F of subsets of X, such that every element of X belongs to at least one subset in F:

$$X = \bigcup_{S \in F} S$$

  We say that a subset $S \in F$ *covers* its elements.
- The problem is to find a minimum-size subset $C \subseteq F$ whose members cover all of X:

$$X = \bigcup_{S \in C} S$$

- We say that any C satisfying the above equation covers X.

8.3. Set Cover problem

- A greedy approximation algorithm for the set cover problem
  - Easily be implemented with polynomial time in |X| and |F|.
  - Since the number of iterations of the loop on lines 3-6 is at most min(|X|, |F|) and the loop body can be implemented to run in time O(|X|.|F|), there is an implementation that runs in time O(|X|.|F|)*min(|X|,|F|) .

```
Greedy-Set-Cover(X, F)
1. U = X
2. C = ∅
3. while U != ∅ do
4.       select an S∈F that maximizes | S ∩ U|
5.       U = U – S
6.       C = C ∪ {S}
7. return C
```

8.3. Set Cover problem

- Given an instance {X, F} of the set cover problem, where X consists of the 12 black points and F = { S_1, S_2, S_3, S_4, S_5, S_6}.
- A minimum size set cover is C* = { S_3, S_4, S_5}.
- The greedy algorithm produces the final set C = {S_1, S_4, S_5, S_3} in order.

S_1 = {x1, x2, x3, x4, x5, x6}

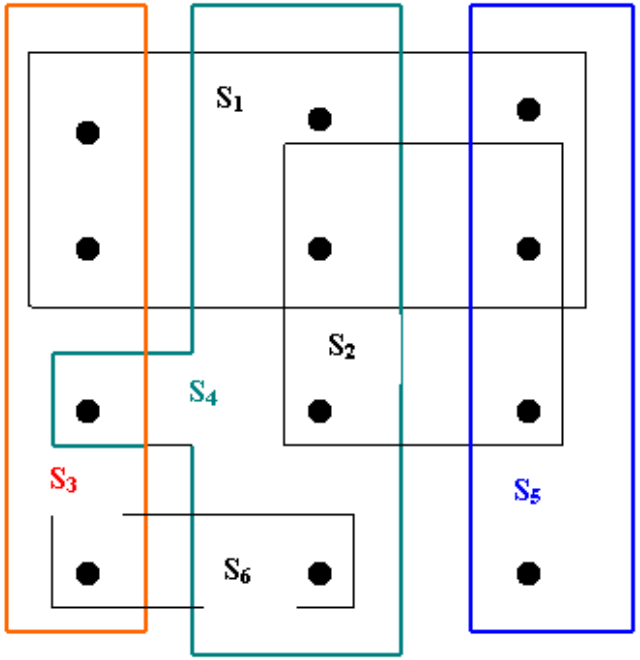
S_2 = {x5, x6, x8, x9}

S_3 = {x1, x4, x7, x10}

S_4 = {x2, x5, x7, x8, x11}

S_5 = {x3, x6, x9, x12}

S_6 = {x10, x11}



8.3. Set Cover problem

- Given an instance {X, F} of the set cover problem, where X consists of the 12 black points and F = { S_1, S_2, S_3, S_4, S_5, S_6}.
- A minimum size set cover is C* = { S_3, S_4, S_5}.
- The greedy algorithm produces the final set C = {S_1, S_4, S_5, S_3} in order.

S_1 = {x1, x2, x3, x4, x5, x6}

S_2 = {x5, x6, x8, x9}

S_3 = {x1, x4, x7, x10}

S_4 = {x2, x5, x7, x8, x11}

S_5 = {x3, x6, x9, x12}

S_6 = {x10, x11}

U = X

Pick S_1: U = {x_7, x_8, x_9, x_{10}, x_{11}, x_{12}}

C = {S_1}

Pick S_4: U = {x_9, x_{10}, x_{12}}

C = {S_1, S_4}

Pick S_5: U = {x_{10}}

C = {S_1, S_4, S_5}

Pick S_3: U = ∅

C = {S_1, S_4, S_5, S_3}

8.3. Set Cover problem

- Ratio bound of the greedy set-cover algorithm
  - Theorem: Greedy-set-cover has a ratio bound H(max{|S|: S $\in$F}).
    - The *d*th harmonic number

$$H_d = \sum_{i=1}^{d} 1/i$$

  - Corollary (Hệ quả): Greedy-set-cover has a ratio bound of (ln|X| +1).

→ Proof is available in:

T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein. *Introduction to Algorithms*. 3rd Edition. The MIT Press, 2009, pp. 1119-1121.

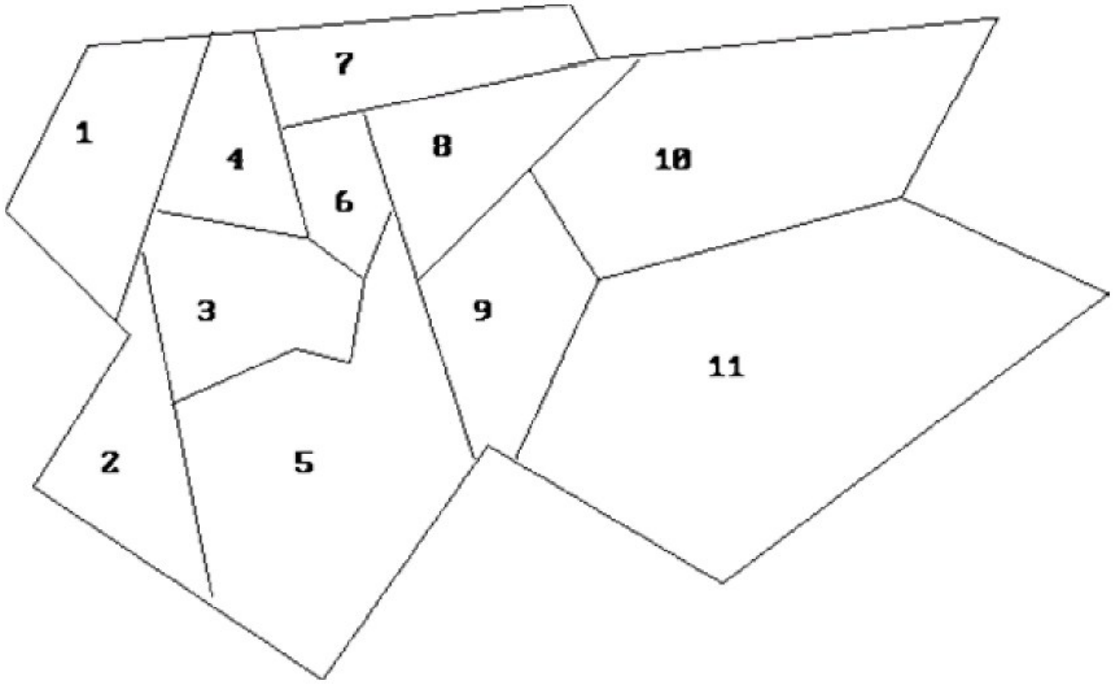
8.3. Set Cover problem

- Applications
  - Assume that X is a set of skills that are needed to solve a problem and we have a set of people available to work on it. We wish to form a team, containing as few people as possible, such that for every requisite skill in X, there is a member in the team having that skill.
  - Assign emergency stations (fire stations) in a city.
  - Allocate sale branch offices for a company.
  - Schedule for bus drivers.

8.3. Set Cover problem

Your turn:

8.3.1. Assign fire stations in a city by solving its corresponding set cover problem. Compare it with the optimal solution: assign fire stations at 3, 8, 9.



The map of a city

8.3. Set Cover problem

Your turn:

- 8.3.2. Given an instance {X, F} of the set cover problem, where X consists of the 11 elements $x_1, x_2,..,x_{11}$ and F = { S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11}. Apply the approximation algorithm to solve the above instance.

$S1 = \{x_1, x_3, x_4\}$
$S2 = \{x_1, x_2, x_4, x_5\}$
$S3 = \{x_1, x_3, x_4, x_5, x_6\}$
$S4 = \{x_1, x_3, x_4, x_6, x_7\}$
$S5 = \{x_2, x_3, x_5, x_8, x_9\}$
$S6 = \{x_1, x_3, x_5, x_6, x_7, x_8\}$
$S7 = \{x_2, x_4, x_6, x_7, x_8\}$
$S8 = \{x_5, x_6, x_7, x_8, x_9, x_{11}\}$
$S9 = \{x_5, x_8, x_{10}, x_{11}\}$
$S10 = \{x_4, x_8, x_9, x_{10}, x_{11}\}$
$S11 = \{x_9, x_{10}, x_{11}\}$

8.3. Set Cover problem

Your turn:

- 8.3.3. Given an instance {X, F} of the set cover problem, where X consists of the 7 elements X = {a, b, c, d, e, f, g} and F = { S1, S2, S3, S4, S5, S6, S7}. Apply the approximation algorithm to solve the above instance.

S1 = {a, b, f, g}

S2 = {a, b, g}

S3 = {a, b, c}

S4 = {e, f, g}

S5 = {f, g}

S6 = {d, f}

S7 = {d}

8.4. Traveling Salesman problem

- Since finding the shortest tour for TSP requires so much computation, we may consider to find a tour that is almost as short as the shortest. That is, it may be possible to find *near-optimal* solution.
- For example: We can use an approximation algorithm for HCP. It's relatively easy to find a tour that is longer by at most a factor of *two* than the optimal tour. The method is based on:
  - the algorithm for finding the *minimum spanning tree*
  - an observation that it is always the cheapest to go directly from vertex *u* to vertex *w*; going by way of any intermediate vertex *v* can't be less expensive.

$$C(u,w) \leq C(u,v) + C(v,w)$$

8.4. Traveling Salesman problem

- APPROX-TSP-TOUR computes a near-optimal tour of an undirected graph G.
  - A *preorder tree walk* recursively visits every vertex in the tree, listing a vertex when it's first encountered, before any of its children is visited.

```
procedure APPROX-TSP-TOUR(G, c);
begin
   select a vertex r ∈ V[G] to be the "root" vertex;
   grow a minimum spanning tree T for G from root r,
   using Prim's algorithm: MST-PRIM(G, c, r);
   apply a preorder tree walk of T and let L be the list
   of vertices visited in the walk;
   form the resulting Hamiltonian cycle H that visits
   the vertices in the order of L;
end
```

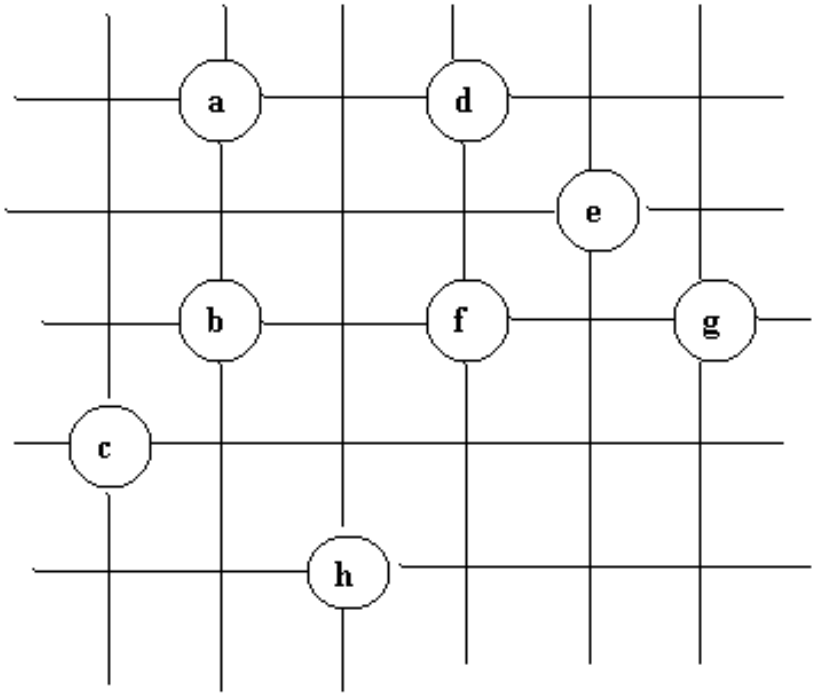
8.4. Traveling Salesman problem

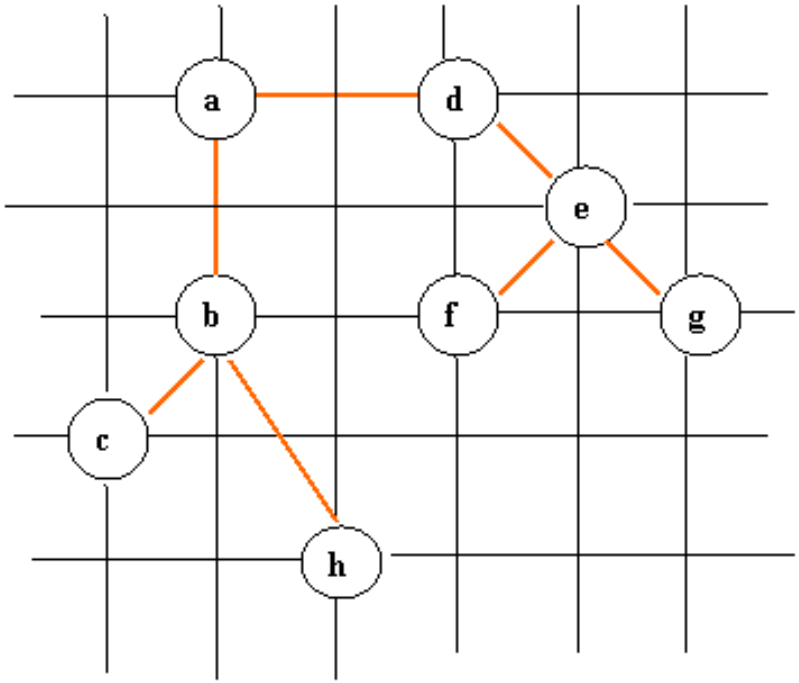
```
procedure MST-PRIM (G, w, r);
/* G = (V, E) is a weighted graph with the weight function
   w, and r is an arbitrary root vertex, p[v] is a parent
   vertex of v. */
begin
   Q := V[G]; /*Q is a priority queue (min-heap)*/
   for each u ∈ Q do key[u] := ∞;
   key[r] := 0; p[r] := NIL;
   while Q is not empty do
   begin
      u := EXTRACT-MIN(Q);
      for each v ∈ Q and w(u, v) < key[v] then
      / * update the key field of vertex v */
      begin
         p[v] := u;  key[v] := w(u, v)
      end
   end
end;
```

Prim's algorithm for a minimum spanning tree

8.4. Traveling Salesman problem

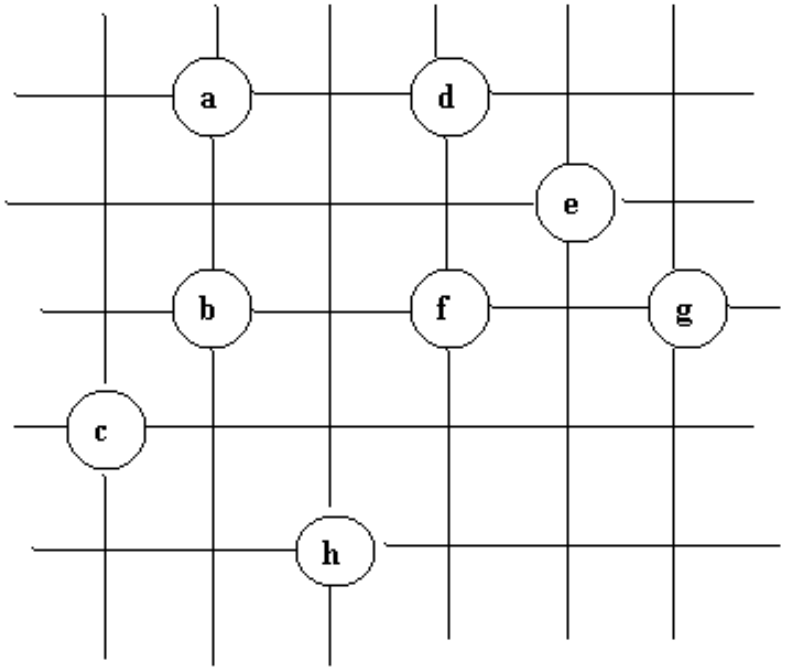
- An example for the traveling salesman problem solved with APPROX-TSP-TOUR





8.4. Traveling Salesman problem

- An example for the traveling salesman problem solved with APPROX-TSP-TOUR

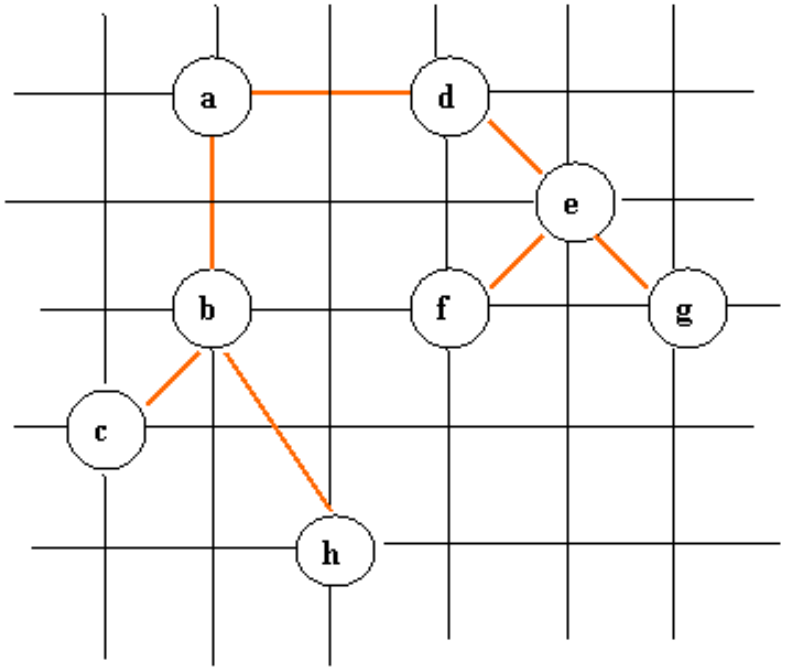


Minimum Spanning Tree Finding with Prim's Algorithm

	a	b	c	d	e	f	g	h
0	**0,nil**	∞	∞	∞	∞	∞	∞	∞
1		**2,a**	$\sqrt{10}$,a	2,a	$\sqrt{10}$,a	$\sqrt{8}$,a	$\sqrt{20}$,a	$\sqrt{17}$,a
2			**$\sqrt{2}$,b**	2,a	$\sqrt{10}$,a	2,b	4,b	$\sqrt{5}$,b
3				**2,a**	$\sqrt{10}$,a	2,b	4,b	$\sqrt{5}$,b
4					**$\sqrt{2}$,d**	2,b	$\sqrt{8}$,d	$\sqrt{5}$,b
5						**$\sqrt{2}$,e**	$\sqrt{2}$,e	$\sqrt{5}$,b
6							**$\sqrt{2}$,e**	$\sqrt{5}$,b
7								**$\sqrt{5}$,b**
T	**0,nil**	**2,a**	**$\sqrt{2}$,b**	**2,a**	**$\sqrt{2}$,d**	**$\sqrt{2}$,e**	**$\sqrt{2}$,e**	**$\sqrt{5}$,b**

8.4. Traveling Salesman problem

- An example for the traveling salesman problem solved with APPROX-TSP-TOUR

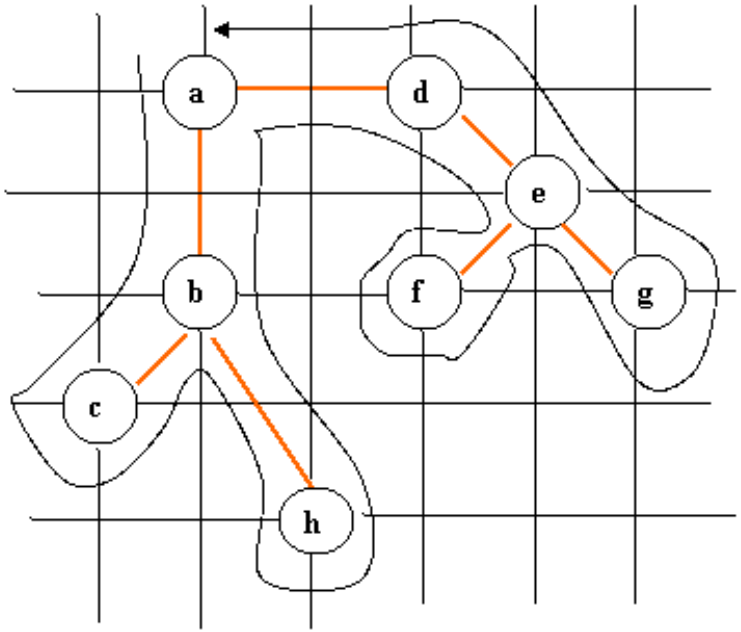


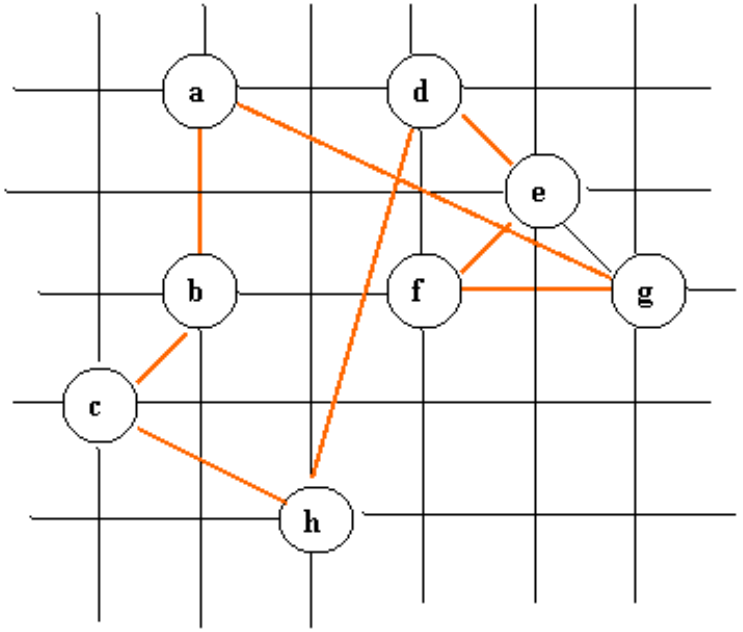
Minimum Spanning Tree Finding with Prim's Algorithm

	a	b	c	d	e	f	g	h
0	**0,nil**	∞	∞	∞	∞	∞	∞	∞
1		**2,a**	√10,a	2,a	√10,a	√8,a	√20,a	√17,a
2			**√2,b**	2,a	√10,a	2,b	4,b	√5,b
3				**2,a**	√10,a	2,b	4,b	√5,b
4					**√2,d**	2,b	√8,d	√5,b
5						**√2,e**	√2,e	√5,b
6							**√2,e**	√5,b
7								**√5,b**
T	**0,nil**	**2,a**	**√2,b**	**2,a**	**√2,d**	**√2,e**	**√2,e**	**√5,b**

8.4. Traveling Salesman problem

- An example for the traveling salesman problem solved with APPROX-TSP-TOUR

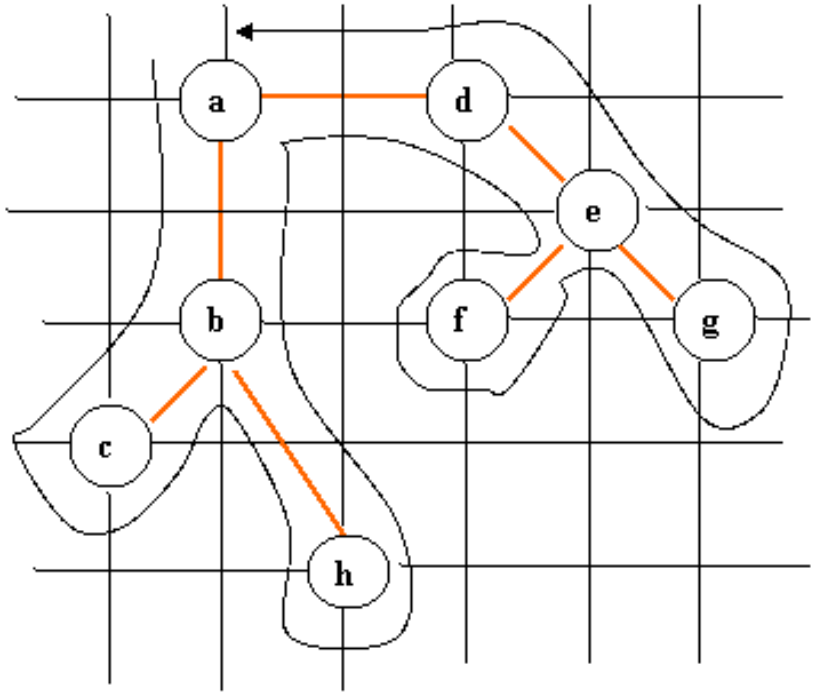


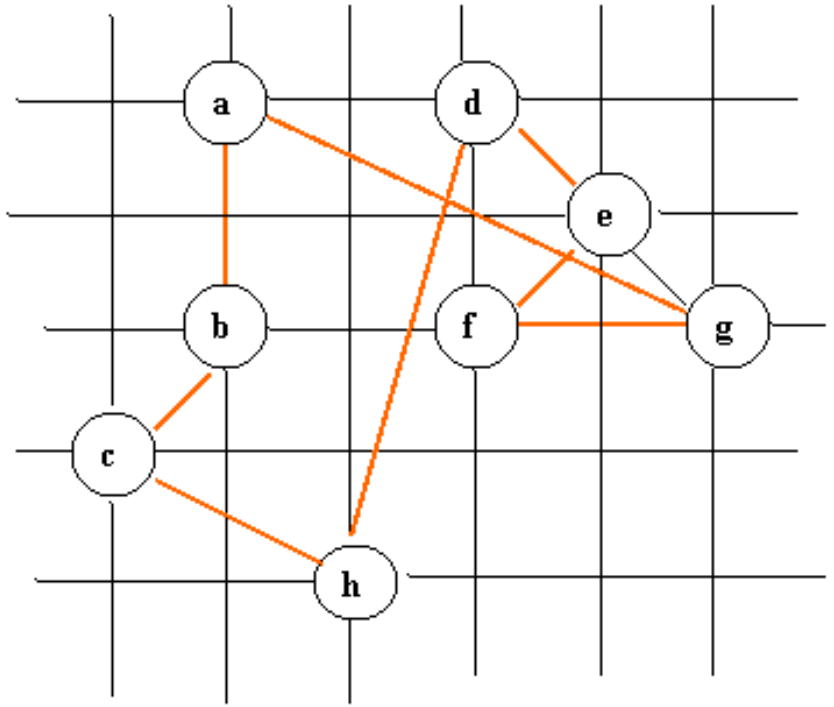


The preorder tree walk is not simple tour, since a node has been visited many times, but it can be fixed, the tree walk visits the vertices in the order *a, b, c, b, h, b, a, d, e, f, e, g, e, d, a.* From this order, we can arrive to the Hamiltonian cycle H: *a, b, c, h, d, e ,f, g, a.*

$$\text{Cost(H)} = 2 + \sqrt{2} + \sqrt{5} + \sqrt{17} + \sqrt{2} + \sqrt{2} + 2 + \sqrt{20} \approx 19.07395$$

8.4. Traveling Salesman problem





The tree walk: *a, b, c, b, h, b, a, d, e, f, e, g, e, d, a.*

The Hamiltonian cycle H: *a, b, c, h, d, e ,f, g, a.*

Triangle inequality:

$c \to h \le c \to b + b \to h$

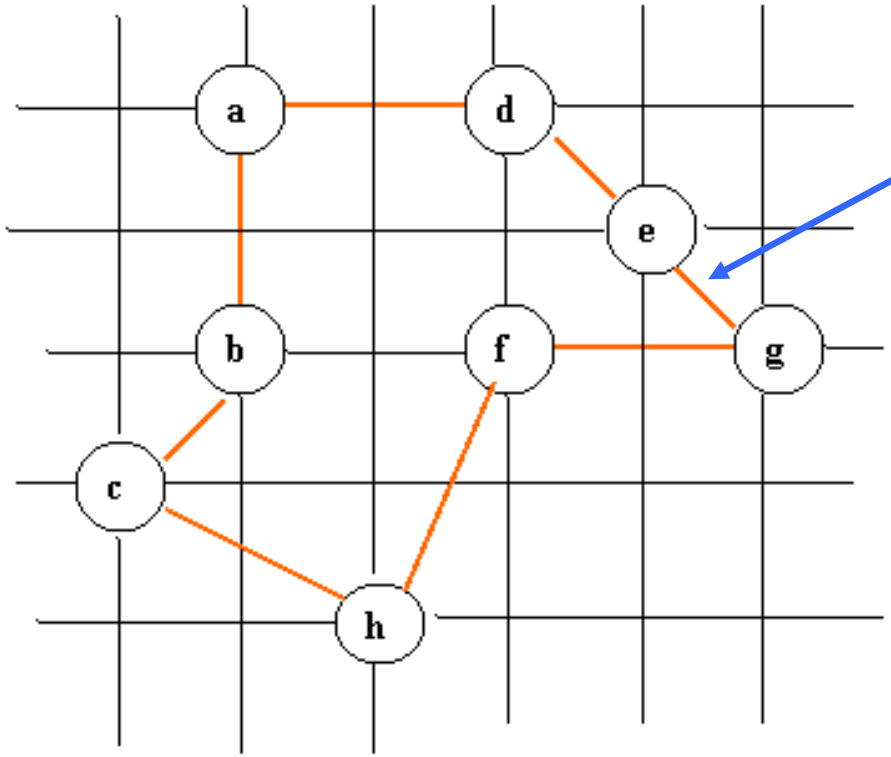
$h \to d \le h \to b + b \to d \le h \to b + b \to a + a \to d$

$f \to g \le f \to e + e \to g$

$g \to a \le g \to e + e \to a \le g \to e + e \to d + d \to a$

8.4. Traveling Salesman problem

- An example for the traveling salesman problem solved with APPROX-TSP-TOUR



The *optimal* tour:

a, b, c, h, f, g, e, d, a

$\text{Cost}(H^*) = 2 + \sqrt{2} + \sqrt{5} + \sqrt{5} + 2 + \sqrt{2} + \sqrt{2} + 2 \approx 14.71477$

The total cost of H is approximately 19.074. An optimal tour H* has the total cost of approximately 14.715.

The running time of APPROX-TSP-TOUR is $O(|V|^2)$ with the main contribution of the running time of MST-PRIM.

8.4. Traveling Salesman problem

- Ratio bound of APPROX-TSP-TOUR
  - Theorem: APPROX-TSP-TOUR is an approximation algorithm with a ratio bound of 2 for the TSP with triangle inequality.
  - Proof: Let H* be an optimal tour for a given set of vertices. Since we obtain a spanning tree by deleting any edge from a tour, if T is a minimum spanning tree for the given set of vertices, then:

    $c(T) \leq c(H^*)$ (1)
  - A full walk W of T traverses every edge of T twice, we have:

    $c(W) = 2c(T)$ (2)

    (1) and (2) imply that:

    $c(W) \leq 2c(H^*)$ (3)

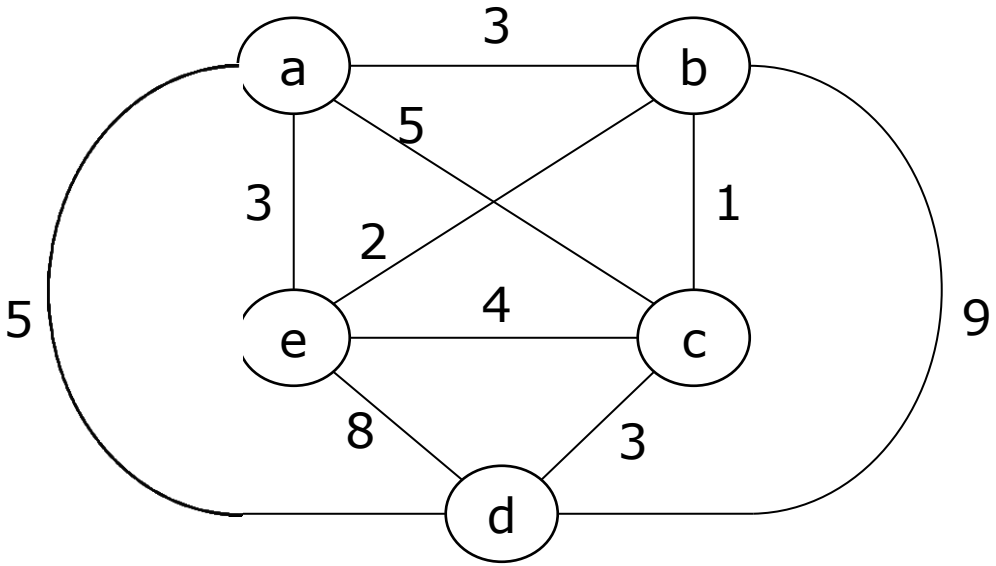
8.4. Traveling Salesman problem

- Ratio bound of APPROX-TSP-TOUR
  - But *W* is not a tour since it visits some vertices more than once. By the triangle inequality, we can delete a visit to any vertex from *W*. By repeatedly applying this operation, we can remove from *W* all but the first visit to each vertex.
  - Let *H* be the cycle corresponding to this preorder walk. It is a Hamiltonian cycle, since every vertex is visited exactly once. Since *H* is obtained by deleting vertices from W, we have: $c(H) \leq c(W)$ (4)
  - From (3) and (4), we conclude: $c(H) \leq 2c(H^*)$.
  - So, APPROX-TSP-TOUR returns a tour whose cost is not more than *twice* the cost of an optimal tour.

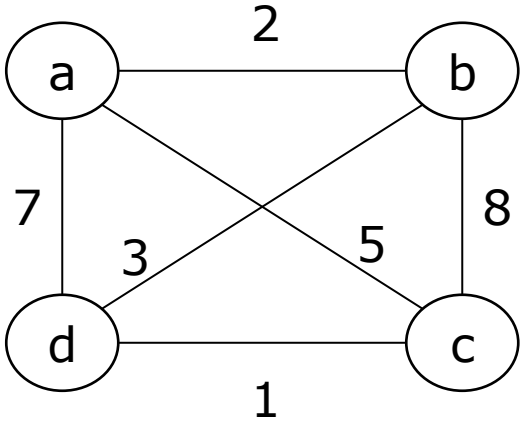
8.4. Traveling Salesman problem

Your turn:

- Given weighted complete graphs, solve the traveling salesman problem with each graph starting from vertex *a* by using the approximation algorithm. What is the tour? How about its found cost?



8.4.1

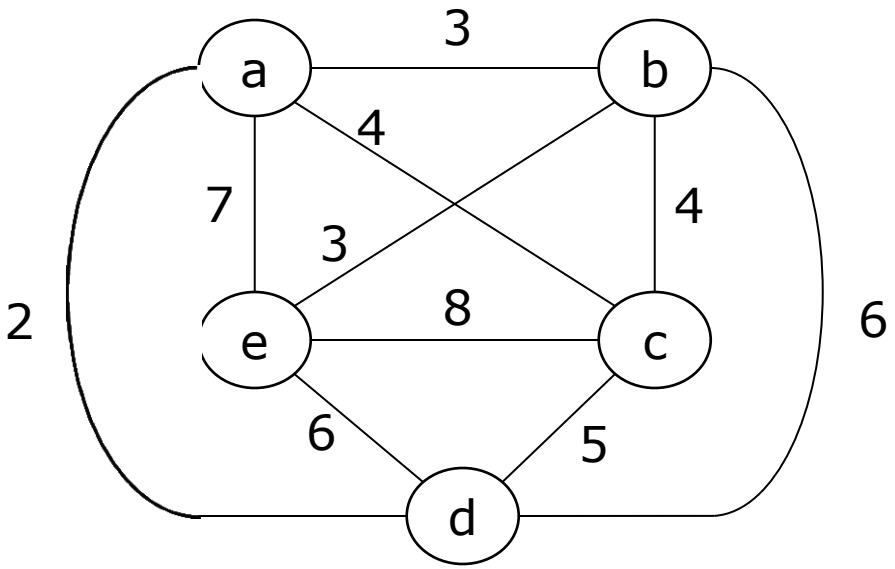


8.4.2

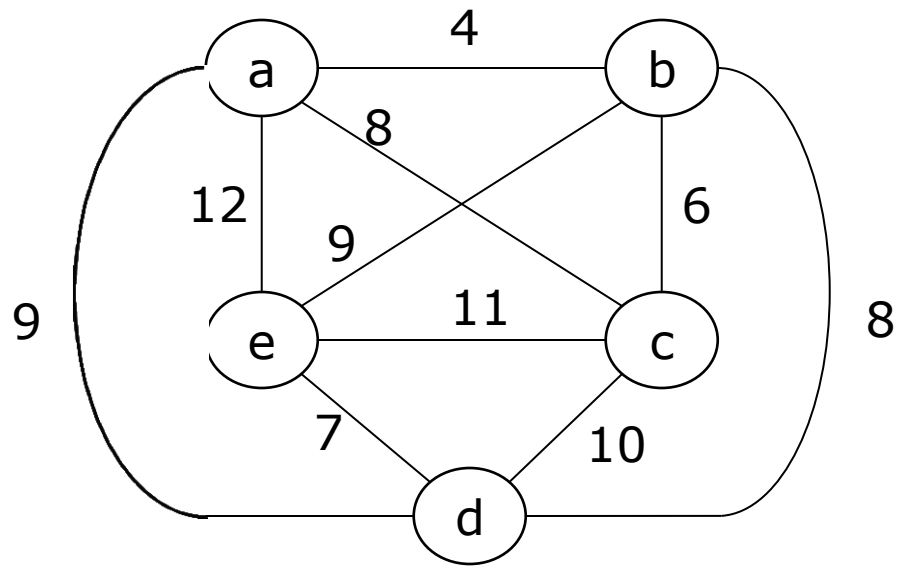
8.4. Traveling Salesman problem

Your turn:

- Given weighted complete graphs, solve the traveling salesman problem with each graph starting from vertex *a* by using the approximation algorithm. What is the tour? How about its found cost?



8.4.3



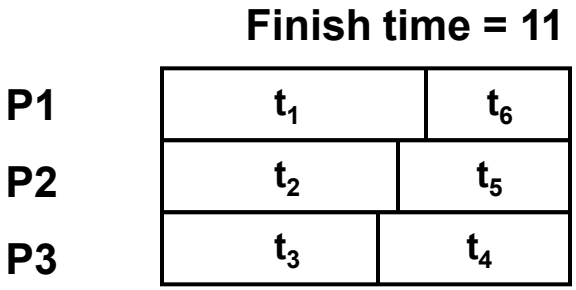
8.4.4

8.5. Scheduling Independent tasks

- An instance of the scheduling problem is defined by a set of n task times, t_i, $1 \leq i \leq n$, and m, the number of processors.
- Obtaining minimum finish time schedules is NP-complete.
- The scheduling rule we will use is called the LPT (longest processing time) rule.
- Definition: An *LPT schedule* is one that is the result of an algorithm, which, whenever a processor becomes free, assigns to that processor a task whose time is the *largest* of those tasks not yet assigned.

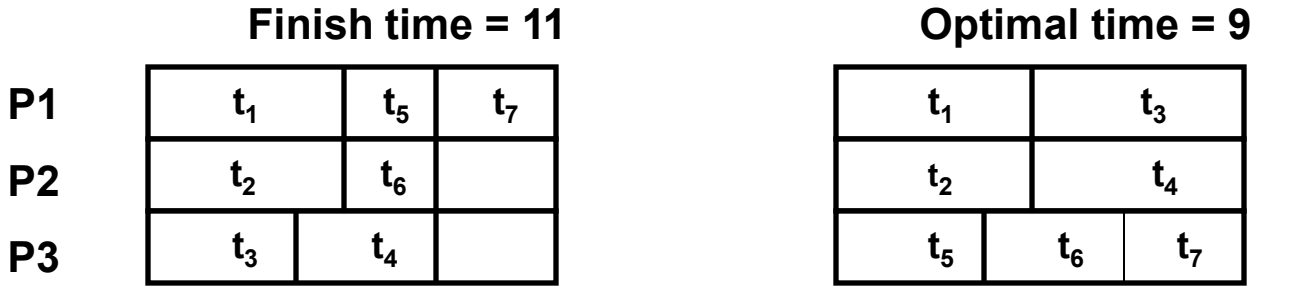
8.5. Scheduling Independent tasks

- Let $m = 3$, $n = 6$ and $(t_1, t_2, t_3, t_4, t_5, t_6) = (8, 7, 6, 5, 4, 3)$. In an LPT schedule tasks 1, 2 and 3 respectively. Tasks 1, 2, and 3 are assigned to processors 1, 2 and 3. Tasks 4, 5 and 6 are respectively assigned to the processors 3, 2, and 1.
- The finish time is 11. Since $\Sigma\, t_i/3 = 11$, the schedule is also optimal.



8.5. Scheduling Independent tasks

- Let $m = 3$, $n = 7$ and $(t_1, t_2, t_3, t_4, t_5, t_6, t_7) = (5, 5, 4, 4, 3, 3, 3)$. The LPT schedule is shown in the following figure. This has a finish time is 11. The optimal schedule is 9. Hence, for this instance $|F^*(I) - F(I)|/F^*(I) = (11-9)/9=2/9$.



(a) LPT schedule

(b) Optimal schedule

8.5. Scheduling Independent tasks

- While the LPT rule may generate optimal schedules for some problem instances, it does not do so for all instances. How bad can LPT schedules be relative to optimal schedules?
- Theorem: Let $F^*(I)$ be the finish time of an optimal m processor schedule for instance I of the task scheduling problem. Let $F(I)$ be the finish time of an LPT schedule for the same instance, then

$$|F^*(I)-F(I)|/F^*(I) \leq 1/3 - 1/(3m)$$

→ The proof of this theorem can be referred to the book

[E. Horowitz and S. Sahni. *Fundamentals of Computer Algorithms.* Pitman Publishing, 1978].

8.5. Scheduling Independent tasks

Your turn:

- 8.5.1. Using the longest processing time (LPT) rule, write and analyze an algorithm for scheduling *n* independent tasks on *m* processors. What is the algorithm design strategy you have used with the LPT rule? Why?

8.5. Scheduling Independent tasks

Your turn:

- 8.5.2. Given the problem of scheduling independent tasks which is defined as follows:

  The number of processors $m = 3$, the set of 6 tasks with $(t_1, t_2, t_3, t_4, t_5, t_6) = (2, 5, 8, 1, 5, 1)$.

Apply the *LPT* rule to solve the above scheduling problem. What is finish time? How different is it from the optimal one?

8.5. Scheduling Independent tasks

Your turn:

- 8.5.3. Given the problem of scheduling independent tasks which is defined below:

  The number of processors $m = 4$, the set of 13 tasks with $(t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10}, t_{11}, t_{12}, t_{13}) = (4, 3, 6, 7, 2, 5, 8, 1, 9, 5, 1, 3, 7)$.

Apply the *LPT* rule to solve the above scheduling problem. What is finish time? How different is it from the optimal one?

8.6. Bin Packing problem

- We are given n objects which have to be placed in bins of equal capacity L.
- Object i requires l_i units of bin capacity.
- The objective is to determine the minimum number of bins needed to accommodate all n objects.
- For example: Let $L = 10$, $n = 6$ and $(l_1, l_2, l_3, l_4, l_5, l_6) = (5, 6, 3, 7, 5, 4)$. How many bins are needed minimally to pack these?
- The bin packing problem is NP-complete.

8.6. Bin Packing problem

- One can derive many simple heuristics for the bin packing problem. In general, they will not obtain optimal packings.
- However, they obtain packings that use only a "small" fraction of bins more than an optimal packing.
- Four heuristics:
  - First fit (FF)
  - Best fit (BF)
  - First fit Decreasing (FFD)
  - Best fit Decreasing (BFD)

8.6. Bin Packing problem

- First-fit
  - Index the bins 1, 2, 3,… All bins are initially filled to level 0. Objects are considered for packing in the order 1, 2, …, n. To pack object i, find the ***least index*** j such that bin j is filled to the level r ($r \leq L - l_i$). Pack i into bin j. Bin j is now filled to level $r + l_i$.
- Best-fit
  - The initial conditions are the same as for FF. When object i is being considered, find the ***least index*** j such that bin j is filled to a level r ($r \leq L - l_i$) and ***r is as large as possible***. Pack i into bin j. Bin j is now filled to level $r + l_i$.

8.6. Bin Packing problem

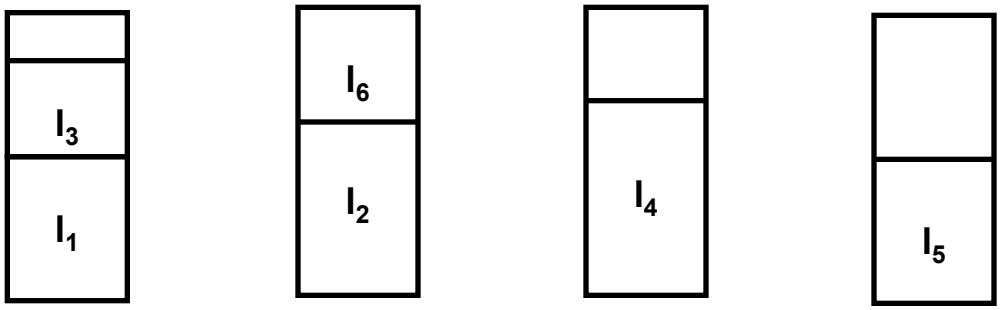
- First-fit decreasing (FFD)
  - Reorder the objects so that $l_i \geq l_{i+1}$, $1 \leq i < n$.

    Now use First-fit to pack the objects.
- Best-fit decreasing (BFD)
  - Reorder the objects so that $l_i \geq l_{i+1}$, $1 \leq i < n$.

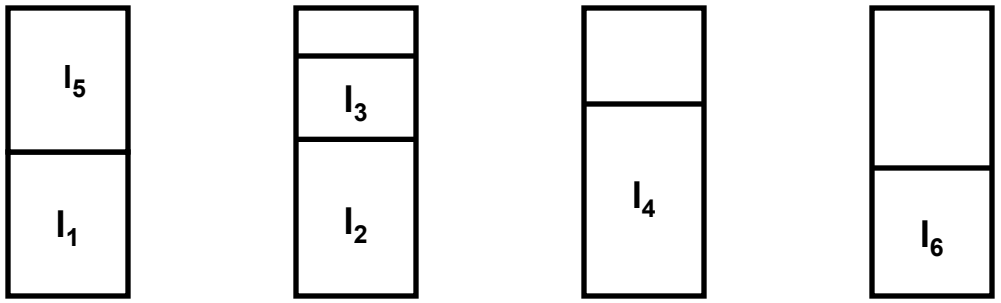
    Now use Best-fit to pack the objects.

8.6. Bin Packing problem

- Given $L = 10$, $n = 6$, and $(l_1, l_2, l_3, l_4, l_5, l_6) = (5, 6, 3, 7, 5, 4)$



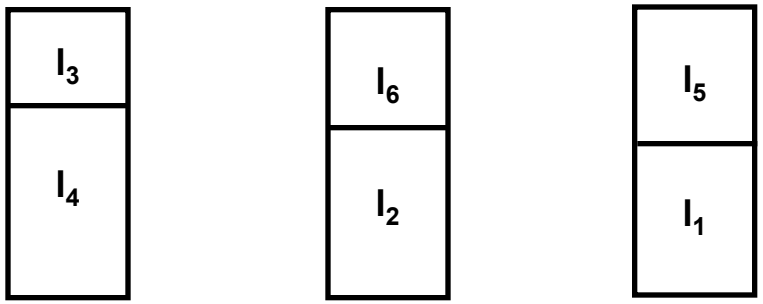
(a) First Fit



(b) Best Fit

8.6. Bin Packing problem

- Given $L = 10$, $n = 6$, and $(l_1, l_2, l_3, l_4, l_5, l_6) = (5, 6, 3, 7, 5, 4)$ → ordered: $(l_4, l_2, l_1, l_5, l_6, l_3)$



(c) First Fit Decreasing and Best Fit Decreasing

FFD and BFD do better than either FF or BF on this instance. While FFD and BFD obtain optimal packings on this example, they do not in general obtain such packings.

8.6. Bin Packing problem

- Theorem:
  - Let I be an instance of the bin packing problem and let F*(I) be the minimum number of bins needed for this instance. The packing generated by either FF or BF uses no more than (17/10)F*(I)+2 bins. The packing generated by either FFD or BFD uses no more than (11/9)F*(I)+4 bins.

→ The proof of this theorem is long and complex, given in [D.S. Johnson. *Near-optimal bin packing algorithms*. PhD Thesis at the Massachusetts Institute of Technology, 1973].

8.6. Bin Packing problem

Your turn:

- 8.6.1. Using four heuristics FF, BF, FFD, and BFD, write and analyze their corresponding algorithms for the bin packing problems on *n* objects with *L* for the capacity of each bin.

8.6. Bin Packing problem

Your turn:

- 8.6.2. Given the bin packing problem in which the capacity of each bin is 13, the number of objects is 8 and the capacity of each object given the array L = (7, 9, 7, 1, 6, 2, 4, 3).

  a) If First Fit is used to solve the problem, what is the result?

  b) If the object with capacity 1 is removed, and First Fit is used, what will be the result?

  c) What is the result if Best Fit is used? First Fit Decreasing? Best Fit Decreasing?

8.6. Bin Packing problem

Your turn:

- 8.6.3. Given the bin packing problem in which the capacity of each bin is 1, the number of objects is 7 and the capacity of each object given the array L = (0.2, 0.6, 0.5, 0.2, 0.8, 0.3, 0.2). Show the results corresponding to the following heuristics.

  a) First Fit

  b) Best Fit

  c) First Fit Decreasing

  d) Best Fit Decreasing

Algorithms Analysis and Design (CO3031)

- Chapter 8. Approximation algorithms
  - 8.1. Why approximation algorithms?
  - 8.2. Vertex Cover problem
  - 8.3. Set Cover problem
  - 8.4. Traveling Salesman problem
  - 8.5. Scheduling Independent tasks
  - 8.6. Bin Packing problem